

- we can also identify the hash-type by a tool called hash-identifier. It is a super easy python tool and will tell you what different type of hashes the one you enter is linked to.

Page No.	
Date:	

Command for the tool: `python3 hash-id.py`

JOHN THE RIPPER: THE BASICS

- Basic syntax: `John [options] [filepath]`

↓
Specifies the options
you want to use

↓
The file containing the
hashes you're trying to
crack and if it's in the
same directory just give
filename.

- Automatic cracking: John can detect what types of ~~passwords~~ hash it has been given and to select appropriate rule and formats to crack it. But it can be unreliable ~~on~~ but if you can't identify hash type it can be a good option.

`John --wordlist=[path to wordlist] [path to file]`

↓
The path to the wordlist

- Format-specific cracking: If you know the hash type, you can tell John to use it while cracking.

Command: `John --format=[format] --wordlist=[path to wordlist]
[path to file]`

↓
The format the hash is in

- NTHASH/NTLM: NT hash is the hash format modern windows operating system machines use to store user and service passwords. In windows SAM (Security account manager) is used to store user account hashed password. We get these hashes by dumping the SAM database using tools like Mimikatz

Format = nt

- Cracking etc/shadow passwords:

Linux stores : etc/shadow → hashed passwords

etc/passwd → usernames

To crack them with John, you must combine them

~~Combine using unshadow~~: Copy the files:

unshadow

etc/passwd and etc/shadow

Combine using ^{un}shadow:

- ① unshadow passwd shadow > combined.txt



Now I will have to send this output as input in John and I need not to specify a mode here as we have made the input specifically for John

- ② John --wordlist=--- combined.txt

NOTE: Combining is not important I can directly give any hash from etc/shadow to John and crack it but the best practice is to combine it with username because John expects input as the input formed by unshadow inside combined.txt.

Now In this step is where John starts cracking.

- Single Crack mode: In this mode John uses only the information provided in the username to try and work out possible ^{passwords} heuristically by slightly changing the letters and numbers ~~contained~~ contained within the username.

Ex: the username is Markus

Possible passwords could be: Markus1, Markus2, Markus, MARKUS, Markus!, Markus\$ etc

- GECOS: stands for general Electric Compshere Operating system. We looked that entries etc/shadow and etc/pass contains fields separated by colon (:). The fifth field is the GECOS field which stores information about the user's full name, office number etc. John take information from this field to add to wordlist when cracking with single crack mode.

What is the syntax for single crack mode?

John --single --format= [path to file]



This is again the combined file of shadow and passwd because we need username info also to run single crack mode. OR I can give a file of the format - username:hash

- What are custom rules?

We know John uses mangling patterns in single crack mode but we can define our own rules, which John use to create passwords. These are important when you have an idea about a password structure of the target.

Custom rules are defined in the ~~john config~~ john.conf file. It is located in etc/john/john.conf.

[List of Rules: HMRules]: The first line is used to define the name of your rule.

The most common modifiers used to tell where the word will be modified:

- **Az**: Take the word and append it with characters you define
- **AD**: " " " " " " prepend " " " " " "
- **cc**: capitalises the character positionally

Now we define what characters should be appended, prepended or otherwise included. We can do this by adding the character set inside the `[]` where ~~the~~ they should be used these should be inside " "

- `[0-9]`: include 0-9
- `[A-Z]`: include A to Z both uppercase and lowercase
- `[A-Z]`: include A to Z only uppercase
- `[a-z]`: include only lowercase
- `[a]`: include only a
- `[!$@%]`: include !, \$, @, %

Ex: assume word `pelopassword` and the target word is `Pelopassword!`

Set of rules: `cAz "[0-9][!$@%]"`

C: capitalise the first letter

Az: ~~Appends~~ Appends to the end of the word

- How to use this rules in the command?

`john --wordlist= --rule= rulename filename`

- How to crack password protected zip files?

We will use a tool name `zip2john` which is used to convert the zip file into a hash format that john can understand and hopefully crack

Page No.
 Date :

Now feed the output file which John will understand as input: Command: John --wordlist = - - - - - outputfile

- Command: `tar -zJohn [tar file] > [Output file]`

New give this output file as input in John for cracking:
Command: `john --wordlist=_____ output file`

- If I want to login into the SSH server using SSH key authentication I should know the password for the private key.

Command: `ssh2john [id_rsa private key file] > [output file]`

Now feed the output file as input in John for cracking
John --wordlist = outputfile

John -- single Combined tit

2 = Wordlist mode : Tries word in list

Command: John --wordlist = file

3 = Incremental mode or brute force :

Tries all combinations a → aa → ab → abc → etc

Command: John --incremental combined.txt

4 = Mask mode or controlled brute force :

Powerful if you know pattern :

Command: John --mask = ?u?l?l?l?l?d?s file

Meaning: 1 uppercase, 4 lowercase, 1 digit, 1 symbol